

Capability scenario	Capability usage
	This capability is required to use the Windows.Media.Devices.AudioDeviceModulesManager class. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved. This is because AudioDeviceModulesManager allows an application to access to all audio effects on a given system. Potentially, the audio effects can be set to negatively impact audio performance on the device.
Background Media Recording	The backgroundMediaRecording capability changes the behavior of the media-specific APIs like the MediaCapture and AudioGraph classes to enable media recording while your app is in the background.
Preview Ink Workspace	The previewInkWorkspace capability allows an app to access the Preview Ink namespace hosted inside the ink workspace. Generally speaking, this is used by an OEM to replace the whiteboard application on a device. This capability is required to the APIs in the Windows.ApplicationModel.Preview.InkWorkspace namespace.
Start Screen Management	The startScreenManagement capability allows apps to silently pin Tiles to the Start screen. Apps can also pin from the background. Not having the startScreenManagement capability does not block any APIs; rather, using startScreenManagement means that the Shell will not display any UI when an app uses the Pin API.
Cortana Permissions	The cortanaPermissions capability allows an app to enumerate the permissions that the user has granted Cortana on the device. The capability also allows an app to grant and revoke Cortana permissions on the device. Note that using cortanaPermissions requires that the device display legal text before granting permissions. As such, it is the responsibility of the app to inform the user of the legal consequences of modifying permissions. This capability is required to gain read access to the HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Search registry settings. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
All App Mods	The allAppMods capability allows an app to access the AppMods folder for all apps. Mod Management utilities use allAppMods to manage mods outside of the game or app that consume them.

Capability scenario	Capability usage
Expanded Resources	The expandedResources capability allows an app access to the Game Mode resources. On Xbox, and on PCs that meet a sufficient bar, Game Mode resources represent a subset of the available CPU cores that are reserved for the app's exclusive use. On Xbox, the app also has exclusive use of a memory partition of at least 4GB. This capability is required to gain exclusive use of CPU and memory resources as defined above.
Protected App	The protectedApp capability grants an app the ability to be loaded into a protected process by the store. When the app is ingested into the store, the store adds a blob to the executable. The store also page signs the executable with a Microsoft key. The process loader checks for this blob rather than the capability to enforce protected process, as the blob needs a Microsoft signature. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Game Monitor	The gameMonitor capability causes the system to use active monitoring to detect game cheats by the app. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
App Diagnostics	The appDiagnostics capability allows an app to get diagnostic information, (such as package information, memory usage, and account name) for any other running UWP app. The information returned includes the domain/machine account name under which the app is running; if the calling app is launched with Administrator rights then the app can retrieve a list of all running apps for all accounts on the machine. This capability is required to use the Windows.System.AppDiagnosticInfo, Windows.System.AppDiagnosticInfo.RequestAppDiagnosticInfoAsync, and Windows.ApplicationModel.AppInfo classes.
Device Portal Providers	The devicePortalProvider capability allows apps to call the Windows.System.Diagnostics.DevicePortal APIs, and serve as a webserver for diagnostic tooling while in Developer Mode. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.

Capability scenario	Capability usage
Enterprise Cloud Single Sign On	The enterpriseCloudSSO capability allows apps to use single sign on with Azure Active Director (AAD) resources inside a hosted web view control.
Automatically accept VoIP calls	The backgroundVoIP capability allows you to automatically receive and accept incoming VoIP calls without requiring the user to accept the call explicitly. Apps utilizing this capability are granted full control of camera and microphone and can use these resources in the background. We don't recommend declaring this capability in apps submitted to the Microsoft Store. For most developers, use of this capability won't be approved.
Reserve resources for VoIP calls	The oneProcessVoIP capability allows you to reserve the CPU and memory resources necessary for a VoIP call in a single-process application. We don't recommend declaring this capability in apps submitted to the Microsoft Store. For most developers, use of this capability won't be approved.
Development Mode Network	The developmentModeNetwork capability allows apps to access network paths using the credentials from the signed-in user when calling the OpenFile Win32 API in a C++/CX UWP app or C++ Windows Runtime component. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Broad Filesystem Access	The broadFileSystemAccess capability allows apps to get the same access to the file system as the user who is currently running the app without any additional file-picker style prompts during runtime. It is important to note that this capability is not required to access files that the user has already chosen using the FilePicker or FolderPicker. This capability works for the Windows.Storage APIs. Because users can grant or deny the permission any time in Settings, you should ensure that your app is resilient to those changes. In the April 2018 update, the default for the permission is On. In the October 2018 update, the default is Off. It is also important that you do not declare any special folder capabilities such as Documents, Pictures, or Videos with this capability. You can enable this capability in your app by adding broadFileSystemAccess to your manifest. For an example, see the File access permissions article. In most cases, to get access to file system locations, your app can use the FileOpenPicker, FileSavePicker, FolderPicker, and FutureAccessList APIs. If you want to request approval to use the <i>broadFileSystemAccess</i> capability, then you must provide specific reasons as to why those APIs aren't sufficient for your needs.

Capability scenario	Capability usage
Note: This capability is not supported on Xbox.	
System Firmware and BIOS	The smbios capability allows apps to access bios data and system firmware data.
Full Trust Permission Level	This is the runFullTrust restricted capability. Terms are defined below, but in short, a package needs this capability if the package uses features for which full trust is needed. A common example is a package that contains one or more full-trust apps. The runFullTrust restricted capability allows a package like that to be installed on a machine. A <i>full trust</i> app is one that sets <code>uap10:TrustLevel</code> to <i>mediumML</i> (see the Application element). A full trust app has a process that runs with an integrity level of <i>medium</i> (see Mandatory Integrity Control). And a <i>package</i> is an <code>.appx</code> or MSIX package (see Building an MSIX package from your code). Another example where this capability is needed is a package with an extension category of <i>windows.firewallRules</i> (see desktop2:Extension). That's considered a full-trust feature; and in that example there's no app to activate, and no process to launch. To use the FullTrustProcessLauncher class, this capability is required, too.
Elevation	The allowElevation restricted capability allows apps that are created by Microsoft partners and enterprises to preserve existing desktop functionality that requires auto-elevation on launch or during an app's lifetime. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved. It will only be approved for line-of-business apps deployed by enterprises to their private store via the Microsoft Store for Business.
Windows Team Device Credentials	The teamEditionDeviceCredential restricted capability allows apps to access APIs that request device account credentials on a Surface Hub device running Windows 10, version 1703 or later. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Windows Team Application View	The teamEditionView restricted capability allows apps to access APIs for hosting an application view on a Surface Hub device running Windows 10, version 1703 or later. We don't recommend that you declare this capability in applications that you

Capability scenario	Capability usage
	submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Camera Processing Extension	The cameraProcessingExtension restricted capability allows apps to process images captured from the camera without direct camera control. This capability is required to call APIs in the Windows.Devices.PointOfService.Provider namespace. Anyone may request access to this capability for store submission.
Data usage Management	The networkDataUsageManagement restricted capability allows apps to gather network data usage information. This capability is required to call GetAttributedNetworkUsageAsync. Anyone may request access to this capability for store submission.
Manage phone line connectivity	The phoneLineTransportManagement capability allows apps to manage system devices responsible for phone line connectivity. This capability is required to use PhoneLineTransportDevice APIs in the Windows.ApplicationModel.Calls namespace.
Unvirtualized Resources	The unvirtualizedResources restricted capability enables your application to declare the RegistryWriteVirtualization and FileSystemWriteVirtualization elements in its package manifest to disable virtualization for the registry and file system. These declarations prevent the system from virtualizing any writes to HKEY_CURRENT_USER or to the user's AppData folder, respectively. This is useful in scenarios where your application expects other applications to read or write the same registry or file system entries as your application. This capability is designed for certain types of desktop PC games that are published by Microsoft and our partners. It's also needed for apps packaged with external location (see Grant package identity by packaging with external location). It is not intended to be used for other scenarios, because it could compromise the system's ability to uninstall cleanly.
Modifiable App	The modifiableApp restricted capability enables your application to declare the windows.mutablePackageDirectories extension in its package manifest. This enables you to provide a name for the folder where your application expects modified or added files to be located. The OS will create this folder and enable your application to use the files in this folder instead of (or in addition to) the files originally installed by the application. This capability is designed for certain types of desktop PC games that are

Capability scenario	Capability usage
	published by Microsoft and our partners. It will not be granted for other scenarios, because it can allow unsigned code to execute.
Package Write Redirection Compatibility Shim	The packageWriteRedirectionCompatibilityShim restricted capability configures your application to create all new files in a per-user location. Any preexisting files opened for writes are first copied into a per-user location and modifications happen to the file in that location. This capability is useful for applications that create or modify files in their installation folder. This capability is designed for certain types of desktop PC games that are published by Microsoft and our partners. However, it might also be applicable to other apps in some cases.
Custom Install Actions	The customInstallActions restricted capability enables your application to declare the windows.customInstall extension in its package manifest so that it can specify one or more additional installer files (.exe or .msi) that are executed with your application. This allows you to specify custom actions for any of the standard deployment scenarios: install, update, repair, or uninstall. For example, this is useful for applications that bundle a 3rd party redistributable component. This capability is designed for certain types of desktop PC games that are published by Microsoft and our partners. It will not be granted for other scenarios.
Packaged Services	The packagedServices restricted capability allows applications that are created by Microsoft partners and enterprises to declare the windows.service extension in its package manifest so that it can install one or more services along with the app. These services can be configured to run under the Local Service, Network Service or Local System accounts. Local Service and Network Service services only require the packagedServices capability. Local System services require both the packagedServices and localSystemServices capabilities. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Local System Services	The localSystemServices restricted capability allows applications that are created by Microsoft partners and enterprises to install one or more Local System services along with the app (that is, your application can declare the StartAccount for the services to be LocalSystem). This scenario also requires the packagedServices capability. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.

Capability scenario	Capability usage
Background Spatial Perception	The backgroundSpatialPerception restricted capability allows an application to access the movement of the user's head, hands, motion controllers, and other tracked objects while the app is running in the background.
UI Access	UIAccess is a feature in Windows that allows certain trusted applications to interact with the user interface (UI) of other applications, even when they are running with higher privileges or in a secure desktop session. This feature is often used by accessibility tools and automation software to provide users with alternative ways to interact with applications. The uiAccess restricted capability must be specified when the <i>uiAccess</i> attribute of the requestedExecutionLevel element is set to true in the app manifest file. For more information see, Security Considerations for Assistive Technologies .

Custom capabilities

The [restricted capabilities](#) section above describes the same capability approval process that you can use to request approval to use a custom capability. The [embedded SIM](#) APIs are examples of APIs that require a custom capability. If you only want to run your application locally in developer mode, then you don't need the custom capability. But you need it to publish your app to the Microsoft Store, or to run it outside of developer mode.

If you have a Windows Technical Account Manager (TAM), then you can work with your TAM to request access. You can find more details at [Contact your Microsoft TAM](#).

To declare a custom capability, modify your [app package manifest](#) source file (`Package.appxmanifest`). Add the **xmlns:uap4** XML namespace declaration, and use the **uap4** prefix when you declare your custom capability. Here's an example.

XML

```
<?xml version="1.0" encoding="utf-8"?>
<Package
  ...
  xmlns:uap4="http://schemas.microsoft.com/appx/manifest/uap/windows10/4">
  ...
  <Capabilities>
    <uap4:CustomCapability
      Name="CompanyName.customCapabilityName_PublisherID"/>
    </Capabilities>
  </Package>
```

ⓘ Note

All **CustomCapability** elements must come after any **Capability** elements and before any **DeviceCapability** elements under the **Capabilities** node in the package manifest.

 Expand table

Capability scenario	Capability usage
App URI handler	An app can register itself as a handler for a given URI, such that when the user opens a link to the specified URI, the app is launched instead of opening the browser. That requires registration on the local machine, and verification by the target web server. The Microsoft.delegatedWebFeatures_8wekyb3d8bbwe custom capability enables an app to host a Progressive Web App (PWA), and to verify that PWA as an app URI handler without referring to the target website for confirmation.
Cellular SAR configuration	The Microsoft.cellularSARConfiguration_8wekyb3d8bbwe custom capability enables an app to perform configuration of a device's specific absorption rate (SAR) properties. See the MobileBroadbandSarManager class.
CoreApplication activation	An app that declares the Microsoft.coreAppActivation_8wekyb3d8bbwe custom capability is CoreApplication-based, but requires to run with full-trust, similar to a Desktop Bridge app.
Custom install actions	Enables the app to use custom install/uninstall actions. An app can use custom actions if it has the customInstallActions restricted capability AND (it is an MSIXVC Xbox Game Pass app, OR it has the Microsoft.classicAppInstaller_8wekyb3d8bbwe custom capability).
ESim management	The Microsoft.eSIMManagement_8wekyb3d8bbwe custom capability enables an app to perform configuration of a device's embedded SIM (eSIM). See the ESim class.
Legacy install behaviors	The Microsoft.classicAppCompat_8wekyb3d8bbwe custom capability protects the case where an app declares a legacy install feature; for example, writing custom COM ProgIds in the Windows Registry.
Machine-wide install	Protects the case where an app declares a legacy install feature that's configured for machine-wide/HKLM registration, which requires-elevation. Specifically required for anything in the app's manifest where the app declares <code>Scope="machine"</code>. For example, an MSIX-based app using the desktop7:ApprovedShellExtension element needs to set scope to machine, and therefore needs to declare the

Capability scenario	Capability usage
	Microsoft.classicAppCompatElevated_8wekyb3d8bbwe custom capability. For more info about that scenario, see desktop7:ApprovedShellExtension .
On-Demand Network	The Microsoft.onDemandHotspotControl_8wekyb3d8bbwe custom capability (the On-Demand Network capability) will be made available only to app developers building end-to-end Wi-Fi hotspot scenarios between a tethering-capable hardware device and a Windows PC. See the WiFiOnDemandHotspotNetwork class.
Registering an approved shell extension	See the Machine-wide install capability scenario.
S-mode	An app that declares the Microsoft.requiresNonSMode_8wekyb3d8bbwe custom capability will be prevented from running on a Windows device in S-mode.
Startup apps	An app can be set up to start automatically at user login. Normally, the user can enable/disable that behavior for each app. The Microsoft.nonUserConfigurableStartupTasks_8wekyb3d8bbwe custom capability configures an app such that the user can't enable/disable its startup behavior.
Windows core 1	An app that declares the Microsoft.deployFullTrustOnHost_8wekyb3d8bbwe custom capability is fully-trusted to use native Win32 APIs on a Windows core device.
Windows core 2	A Desktop Bridge app that declares the Microsoft.notSupportedInCoreV1_8wekyb3d8bbwe custom capability will be prevented from running on a Windows core device.
Windows Hello companion device framework	The functionality related to the Microsoft.secondaryAuthenticationFactorForLogon_8wekyb3d8bbwe custom capability is deprecated as of Windows 10, version 2004 (10.0; Build 19041), and we no longer grant this capability to developer accounts. An app that declares this capability can be registered with the companion device framework to enable <i>Windows unlock</i> with the app's associated companion device. For more info, see Windows Unlock with Windows Hello companion (IoT) devices .

Related topics

- [Submission options](#)
- [How to specify capabilities in a package manifest](#)
- [How to specify device capabilities in a package manifest](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Set up automated builds for your UWP app

Article • 10/20/2022

You can use Azure Pipelines to create automated builds for UWP projects. In this article, we'll look at different ways to do this. We'll also show you how to perform these tasks by using the command line so that you can integrate with any other build system.

Create a new Azure Pipeline

Begin by [signing up for Azure Pipelines](#) if you haven't done so already.

Next, create a pipeline that you can use to build your source code. For a tutorial about building a pipeline to build a GitHub repository, see [Create your first pipeline](#). Azure Pipelines supports the repository types listed [in this article](#).

Set up an automated build

We'll start with the default UWP build definition that's available in Azure Dev Ops and then show you how to configure the pipeline.

In the list of build definition templates, choose the **Universal Windows Platform** template.

✓ Connect

✓ Select

Configure

Create pipeline

New pipeline

Configure your pipeline



ASP.NET Core recommended

Build and test ASP.NET Core projects targeting .NET Core.



ASP.NET

Build and test ASP.NET projects.



ASP.NET Core (.NET Framework)

Build and test ASP.NET Core projects targeting the full .NET Framework.



Universal Windows Platform

Build a Universal Windows Platform project using Visual Studio.

This template includes the basic configuration to build your UWP project:

```
yml

trigger:
- master

pool:
  vmImage: 'windows-latest'

variables:
  solution: '**/*.sln'
  buildPlatform: 'x86|x64|ARM'
  buildConfiguration: 'Release'
  appxPackageDir: '$(build.artifactStagingDirectory)\AppxPackages\'

steps:
- task: NuGetToolInstaller@1

- task: NuGetCommand@2
  inputs:
    restoreSolution: '$(solution)'

- task: VSBUILD@1
  inputs:
    platform: 'x86'
    solution: '$(solution)'
    configuration: '$(buildConfiguration)'
    msbuildArgs: '/p:AppxBundlePlatforms="$(buildPlatform)"
/p:AppxPackageDir="$(appxPackageDir)" /p:AppxBundle=Always
/p:UapAppxPackageBuildMode=StoreUpload'
```

The default template tries to sign the package with the certificate specified in the .csproj file. If you want to sign your package during the build you must have access to the private key. Otherwise, you can disable signing by adding the parameter

`/p:AppxPackageSigningEnabled=false` to the `msbuildArgs` section in the YAML file.

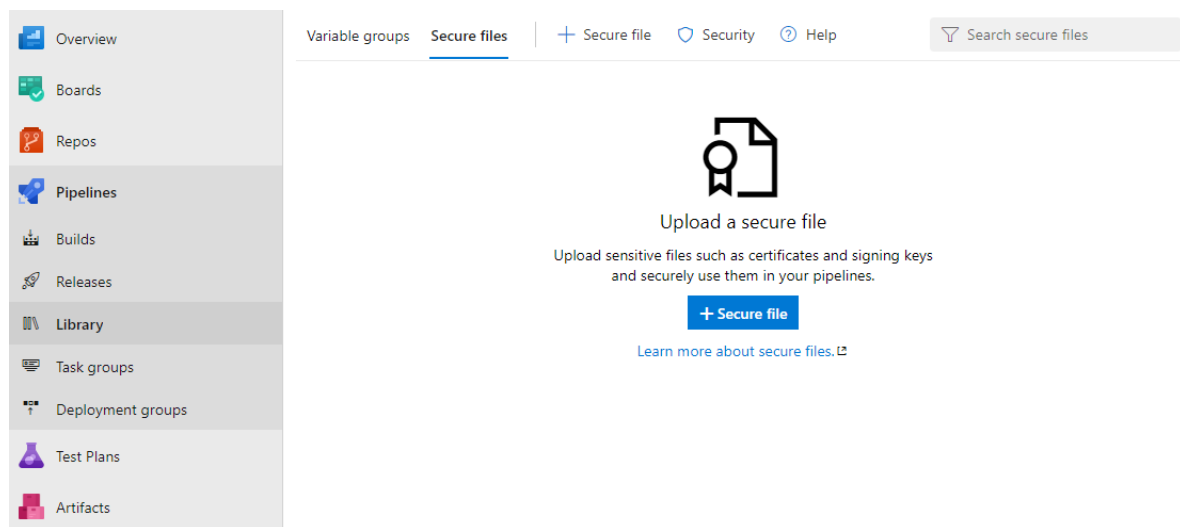
Add your project certificate to the Secure files library

You should avoid submitting certificates to your repo if at all possible, and git ignores them by default. To manage the safe handling of sensitive files like certificates, Azure DevOps supports the [secure files](#) feature.

To upload a certificate for your automated build:

1. In Azure Pipelines, expand **Pipelines** in the navigation pane and click **Library**.

2. Click the **Secure files** tab and then click **+ Secure file**.



3. Browse to the certificate file and click **OK**.

4. After you upload the certificate, select it to view its properties. Under **Pipeline permissions**, enable the **Authorize for use in all pipelines** toggle.

Pipeline permissions

☒ Authorize for use in all pipelines

5. If the private key in the certificate has a password, we recommend that you store your password in [Azure Key Vault](#) and then link the password to a [variable group](#). You can use the variable to access the password from the pipeline. Note that a password is only supported for the private key; using a certificate file that is itself password-protected is not currently supported.

ⓘ Note

Starting in Visual Studio 2019, a temporary certificate is no longer generated in UWP projects. To create or export certificates, use the PowerShell cmdlets described in [this article](#).

Configure the Build solution build task

This task compiles any solution that's in the working folder to binaries and produces the output app package file. This task uses MSBuild arguments. You'll have to specify the value of those arguments. Use the following table as a guide.

MSBuild argument	Value	Description
------------------	-------	-------------

MSBuild argument	Value	Description
AppxPackageDir	\$(Build.ArtifactStagingDirectory)\AppxPackages	Defines the folder to store the generated artifacts.
AppxBundlePlatforms	\$(Build.BuildPlatform)	Enables you to define the platforms to include in the bundle.
AppxBundle	Always	Creates an .msixbundle/.appxbundle with the .msix/.appx files for the platform specified.
UapAppxPackageBuildMode	StoreUpload	Generates the .msixupload/.appxupload file and the _Test folder for sideloading.
UapAppxPackageBuildMode	CI	Generates the .msixupload/.appxupload file only.
UapAppxPackageBuildMode	SideloadOnly	Generates the _Test folder for sideloading only.
AppxPackageSigningEnabled	true	Enables package signing.
PackageCertificateThumbprint	Certificate Thumbprint	This value must match the thumbprint in the signing certificate, or be an empty string.
PackageCertificateKeyFile	Path	The path to the certificate to use. This is retrieved from the secure file metadata.
PackageCertificatePassword	Password	The password for the private key in the certificate. We recommend that you store your password in Azure Key Vault and link the password to variable group . You can pass the variable to this argument.

Configure the build

If you want to build your solution by using the command line, or by using any other build system, run MSBuild with these arguments.

PowerShell

```
/p:AppxPackageDir="$(Build.ArtifactStagingDirectory)\AppxPackages\\"
/p:UapAppxPackageBuildMode=StoreUpload
/p:AppxBundlePlatforms="$(Build.BuildPlatform)"
/p:AppxBundle=Always
```

Configure package signing

To sign the MSIX (or .appx) package the pipeline needs to retrieve the signing certificate. To do this, add a DownloadSecureFile task prior to the VSBuild task. This will give you access to the signing certificate via `signingCert`.

yml

```
- task: DownloadSecureFile@1
  name: signingCert
  displayName: 'Download CA certificate'
  inputs:
    secureFile: '[Your_Pfx].pfx'
```

Next, update the VSBuild task to reference the signing certificate:

yml

```
- task: VSBuild@1
  inputs:
    platform: 'x86'
    solution: '$(solution)'
    configuration: '$(buildConfiguration)'
    msbuildArgs: '/p:AppxBundlePlatforms="$(buildPlatform)"
                  /p:AppxPackageDir="$(appxPackageDir)"
                  /p:AppxBundle=Always
                  /p:UapAppxPackageBuildMode=StoreUpload
                  /p:AppxPackageSigningEnabled=true
                  /p:PackageCertificateThumbprint=""

                  /p:PackageCertificateKeyFile="$(signingCert.secureFilePath)''
```

ⓘ Note

The PackageCertificateThumbprint argument is intentionally set to an empty string as a precaution. If the thumbprint is set in the project but does not match the signing certificate, the build will fail with the error: Certificate does not match supplied signing thumbprint.

Review parameters

The parameters defined with the `$( )` syntax are variables defined in the build definition, and will change in other build systems.

[Build](#) [Options](#) [Repository](#) [Variables](#) [Triggers](#) [General](#) [Retention](#) [History](#)

 Save  Undo

List of predefined variables

Name	Value	Allow at Queue Time
system.collectionId	a4bc4c27-208c-44ef-a06f-76a258c417b4	☐
system.teamProject	UWPDemos	☐
system.definitionId	20	☐
✕ BuildConfiguration	release	☒
✕ BuildPlatform	x86 x64 ARM	☒
✕ system.debug	false	☒

 Add variable 

To view all predefined variables, see [Predefined build variables](#).

Configure the Publish Build Artifacts task

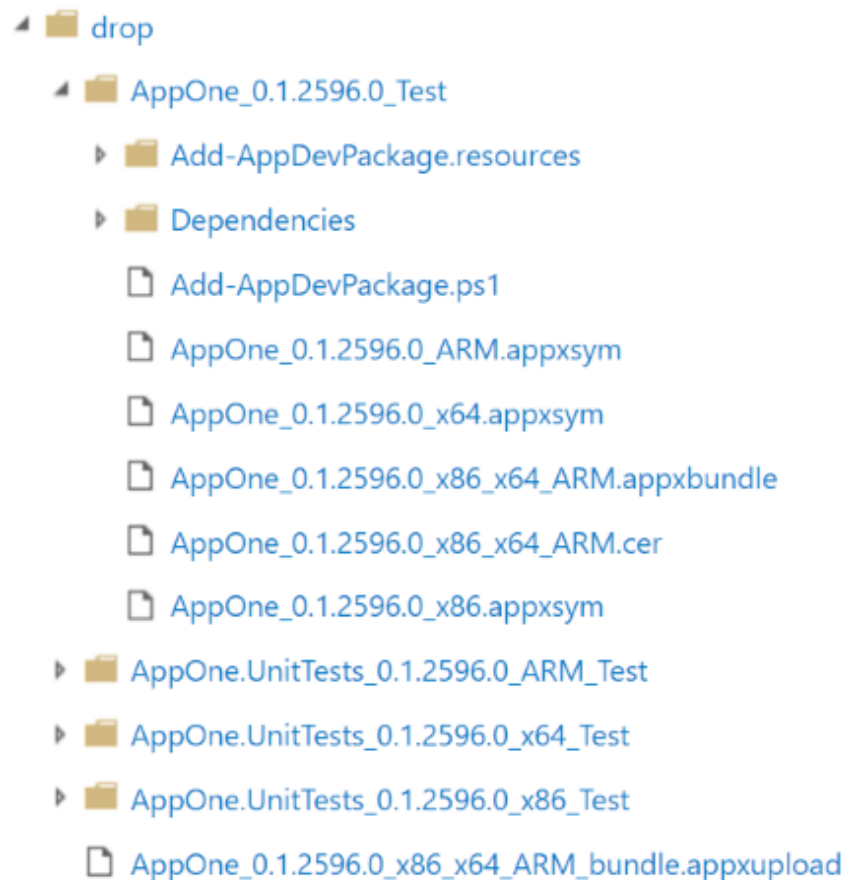
The default UWP pipeline does not save the generated artifacts. To add the publish capabilities to your YAML definition, add the following tasks.

```
yml
- task: CopyFiles@2
  displayName: 'Copy Files to: $(build.artifactstagingdirectory)'
  inputs:
    SourceFolder: '$(system.defaultworkingdirectory)'
    Contents: '**\bin\$(BuildConfiguration)\**'
    TargetFolder: '$(build.artifactstagingdirectory)'

- task: PublishBuildArtifacts@1
  displayName: 'Publish Artifact: drop'
  inputs:
    PathtoPublish: '$(build.artifactstagingdirectory)'
```


You can see the generated artifacts in the **Artifacts** option of the build results page.

Artifacts Explorer



Because we've set the `UapAppxPackageBuildMode` argument to `StoreUpload`, the artifacts folder includes the package for submission to the Store (`.msixupload/.appxupload`). Note that you can also submit a regular app package (`.msix/.appx`) or an app bundle (`.msixbundle/.appxbundle/`) to the Store. For the purposes of this article, we'll use the `.appxupload` file.

Address bundle errors

If you add more than one UWP project to your solution and then try to create a bundle, you might receive an error like this one.

```
MakeAppx(0,0): Error : Error info: error 80080204: The package with file name
"AppOne.UnitTests_0.1.2595.0_x86.appx" and package full name "8ef641d1-4557-4e33-
957f-6895b122f1e6_0.1.2595.0_x86__scrj5wvaadc6" is not valid in the bundle because
it has a different package family name than other packages in the bundle
```

This error appears because at the solution level, it's not clear which app should appear in the bundle. To resolve this issue, open each project file and add the following properties at the end of the first `<PropertyGroup>` element.

Project	Properties
App	<code><AppxBundle>Always</AppxBundle></code>
UnitTests	<code><AppxBundle>Never</AppxBundle></code>

Then, remove the `AppxBundle` MSBuild argument from the build step.

Related topics

- [Build your .NET app for Windows](#)
- [Packaging UWP apps](#)
- [Sideload LOB apps in Windows](#)
- [Create a certificate for package signing](#)

Install apps with the WinAppDeployCmd.exe tool

Article • 10/20/2022

Windows Application Deployment (WinAppDeployCmd.exe) is a command line tool that can use to deploy a Universal Windows Platform (UWP) app from a Windows 10 PC to any Windows 10 device. You can use this tool to deploy an app package when the Windows 10 device is connected by USB or available on the same subnet without needing Microsoft Visual Studio or the solution for that app. You can also deploy the app without packaging first to a remote PC or Xbox One. This article describes how to install UWP apps using this tool.

You just need the Windows 10 SDK installed to run the WinAppDeployCmd tool from a command prompt or a script file. When you install an app with WinAppDeployCmd.exe, this uses the .appx/.msix file or AppxManifest(for loose files) to side-load your app onto a Windows 10 device. This command does not install the certificate required for your app. To run the app, the Windows 10 device must be in developer mode or already have the certificate installed.

To deploy to mobile devices, you must first create a package. For more information, see [here](#).

The **WinAppDeployCmd.exe** tool is located here on your Windows 10 PC: **C:\Program Files (x86)\Windows Kits\10\bin\<SDK Version>\x86\WinAppDeployCmd.exe** (based on your installation path for the SDK).

ⓘ Note

In version 15063 and later of the SDK, the SDK is installed side by side within version-specific folders. Previous SDKs (prior to and including 14393) are written directly to the parent folder.

First, connect your Windows 10 device to the same subnet or connect it directly to your Windows 10 machine with a USB connection. Then use the following syntax and examples of this command later in this article to deploy your UWP app:

WinAppDeployCmd syntax and options

This is the general syntax used for **WinAppDeployCmd.exe**:

CMD

```
WinAppDeployCmd command -option <argument>
```

Here are some additional syntax examples for using various commands:

CMD

```
WinAppDeployCmd devices
WinAppDeployCmd devices <x>
WinAppDeployCmd install -file <path> -ip <address>
WinAppDeployCmd install -file <path> -guid <address> -pin <p>
WinAppDeployCmd install -file <path> -ip <address> -dependency <a> <b>
WinAppDeployCmd install -file <path> -guid <address> -dependency <a> <b>
WinAppDeployCmd uninstall -file <path>
WinAppDeployCmd uninstall -package <name>
WinAppDeployCmd update -file <path>
WinAppDeployCmd list -ip <address>
WinAppDeployCmd list -guid <address>
WinAppDeployCmd deployfiles -file <path> -remotedeploydir
<remoterelativepath> -ip <address>
WinAppDeployCmd registerfiles -remotedeploydir <remoterelativepath> -ip
<address>
WinAppDeployCmd addcreds -credserver <server> -credusername <username> -
credpassword <password> -ip <address>
WinAppDeployCmd getcreds -credserver <server> -ip <address>
WinAppDeployCmd deletcreds -credserver <server> -ip <address>
```

You can install or uninstall an app on the target device, or you can update an app that's already installed. To keep data or settings saved by an app that's already installed, use the **update** options instead of the **install** options.

The following table describes the commands for **WinAppDeployCmd.exe**.

Command	Description
devices	Show the list of available network devices.
install	Install a UWP app package to the target device.
update	Update a UWP app that is already installed on the target device.
list	Show the list of UWP apps installed on the specified target device.
uninstall	Uninstall the specified app package from the target device.
deployfiles	Copy over loose file app at the target path to the remote relative path on the device.
registerfiles	Register the loose file app at the remote deploy directory.

Command	Description
addcreds	Add credentials to an Xbox to allow it to access a network location for app registration.
getcreds	Get network credentials for the target uses when running an application from a network share.
deletecreds	Delete network credentials the target uses when running an application from a network share.

The following table describes the options for **WinAppDeployCmd.exe**.

Command	Description
-h (-help)	Show the commands, options and arguments.
-ip	IP address of the target device.
-g (-guid)	Unique identifier of the target device.
-d (-dependency)	(Optional) Specifies the dependency path for each of the package dependencies. If no path is specified, the tool searches for dependencies in the root directory for the app package and the SDK directories.
-f (-file)	File path for the app package to install, update or uninstall.
-p (-package)	The full package name for the app package to uninstall. (You can use the list command to find the full names for packages already installed on the device)
-pin	A pin if it is required to establish a connection with the target device. (You will be prompted to retry with the -pin option if authentication is required)
-credserver	The server name of the network credentials for use by the target.
-credusername	The user name of the network credentials for use by the target.
-credpassword	The password of the network credentials for use by the target.
-connecttimeout	The timeout in seconds used when connecting to the device.
-remotedeploydir	Relative directory path/name to copy files over to on the remote device; This will be a well-known, automatically determined remote deployment folder.
-deleteextrafile	Switch to indicate whether existing files in the remote directory should be purged to match the source directory.

The following table describes the options for **WinAppDeployCmd.exe**.

Argument	Description
----------	-------------

Argument	Description
<x>	Timeout in seconds. (Default is 10)
<address>	IP address or unique identifier of the target device.
<a> ...	Dependency path for each of the app package dependencies.
<p>	An alpha-numeric pin shown in the device settings to establish a connection.
<path>	File system path.
<name>	Full package name for the app package to uninstall.
<server>	Server on the file network.
<username>	User for the credentials with access to the server on the file network.
<password>	Password for the credentials with access to the server on the files network.
<remotedeploydir>	Directory on device relative to the deployment location

WinAppDeployCmd.exe examples

Here are some examples of how to deploy from the command-line using the syntax for **WinAppDeployCmd.exe**.

Shows the devices that are available for deployment. The command times out in 3 seconds.

CMD

```
WinAppDeployCmd devices 3
```

Installs the app from MyApp.appx package that is in your PC's Downloads directory to a Windows 10 device with an IP address of 192.168.0.1 with a PIN of A1B2C3 to establish a connection with the device

CMD

```
WinAppDeployCmd install -file "Downloads\MyApp.appx" -ip 192.168.0.1 -pin A1B2C3
```

Uninstalls the specified package (based on its full name) from a Windows device with an IP address of 192.168.0.1. You can use the list command to see the full names of any

packages that are installed on a device.

CMD

```
WinAppDeployCmd uninstall -package Company.MyApp_1.0.0.1_x64__qwertyuiop -ip 192.168.0.1
```

Updates the app that is already installed on the Windows 10 device with an IP address of 192.168.0.1 using the specified app package.

CMD

```
WinAppDeployCmd update -file "Downloads\MyApp.appx" -ip 192.168.0.1
```

Deploys the files of an app to a PC or Xbox with an IP address of 192.168.0.1 in the same folder as the AppxManifest to the app1_F5 directory under the deployment path of the device.

CMD

```
WinAppDeployCmd deployfiles -file "C:\apps\App1\AppxManifest.xml" -remotedeploydir app1_F5 -ip 192.168.0.1
```

Registers the app at the app1_F5 directory under the deployment path of the PC or Xbox at 192.168.0.1.

CMD

```
WinAppDeployCmd registerfiles -remotedeploydir app1_F5 -ip 192.168.0.1
```

Using WinAppDeployCmd to set up Run from PC deployment on Xbox One

Run from PC allows you to deploy a UWP application to an Xbox One without copying the binaries over, instead the binaries are hosted on a network share on the same network as the Xbox. In order to do this, you need a developer unlocked Xbox One, and a loose file UWP application on a network drive that the Xbox can access.

Run this to register the app:

CMD

```
WinAppDeployCmd registerfiles -ip <Xbox One IP> -remotedeploydir <location  
of app> -username <user for network> -password <password for user>
```

```
ex. WinAppDeployCmd register files -ip 192.168.0.1 -remotedeploydir  
\\driveA\myAppLocation -username admin -password A1B2C3
```


Download and install package updates from the Store

Article • 11/01/2022

Starting in Windows 10, version 1607, you can use methods of the [StoreContext](#) class in the [Windows.Services.Store](#) namespace to programmatically check for package updates for the current app from the Microsoft Store, and download and install the updated packages. You can also query for packages that you have marked as mandatory in Partner Center and disable functionality in your app until the mandatory update is installed.

Additional [StoreContext](#) methods introduced in Windows 10, version 1803 enable you to download and install package updates silently (without displaying a notification UI to the user), uninstall an [optional package](#), and get info about packages in the download and install queue for your app.

These features help you automatically keep your user base up to date with the latest version of your app, optional packages, and related services in the Store.

Download and install package updates with the user's permission

This code example demonstrates how to use the [GetAppAndOptionalStorePackageUpdatesAsync](#) method to discover all available package updates from the Store and then call the [RequestDownloadAndInstallStorePackageUpdatesAsync](#) method to download and install the updates. When using this method to download and install updates, the OS displays a dialog that asks the user's permission before downloading the updates.

ⓘ Note

These methods support required and [optional packages](#) for your app. Optional packages are useful for downloadable content (DLC) add-ons, dividing your large app for size constraints, or for shipping additional content separate from your core app. To get permission to submit an app that uses optional packages (including DLC add-ons) to the Store, see [Windows developer support](#).

This code example assumes:

- The code runs in the context of a [Page](#).
- The **Page** contains a [ProgressBar](#) named `downloadProgressBar` to provide status for the download operation.
- The code file has a **using** statement for the **Windows.Services.Store**, **Windows.Threading.Tasks**, and **Windows.UI.Popups** namespaces.
- The app is a single-user app that runs only in the context of the user that launched the app. For a [multi-user app](#), use the [GetForUser](#) method to get a **StoreContext** object instead of the [GetDefault](#) method.

C#

```
private StoreContext context = null;

public async Task DownloadAndInstallAllUpdatesAsync()
{
    if (context == null)
    {
        context = StoreContext.GetDefault();
    }

    // Get the updates that are available.
    IReadOnlyList<StorePackageUpdate> updates =
        await context.GetAppAndOptionalStorePackageUpdatesAsync();

    if (updates.Count > 0)
    {
        // Alert the user that updates are available and ask for their
        // consent to start the updates.
        MessageDialog dialog = new MessageDialog(
            "Download and install updates now? This may cause the
            application to exit.", "Download and Install?");
        dialog.Commands.Add(new UICommand("Yes"));
        dialog.Commands.Add(new UICommand("No"));
        IUICommand command = await dialog.ShowAsync();

        if (command.Label.Equals("Yes",
            StringComparison.CurrentCultureIgnoreCase))
        {
            // Download and install the updates.
            IAsyncOperationWithProgress<StorePackageUpdateResult,
            StorePackageUpdateStatus> downloadOperation =
                context.RequestDownloadAndInstallStorePackageUpdatesAsync(updates);

            // The Progress async method is called one time for each step in
            // the download and installation process for each package in this request.
            downloadOperation.Progress = async (asyncInfo, progress) =>
            {
                await
```

```

this.Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>
    {
        downloadProgressBar.Value =
progress.PackageDownloadProgress;
    });
};

StorePackageUpdateResult result = await
downloadOperation.AsTask();
}
}
}

```

ⓘ Note

To only download (but not install) the available package updates, use the [RequestDownloadStorePackageUpdatesAsync](#) method.

Display download and install progress info

When you call the [RequestDownloadStorePackageUpdatesAsync](#) or [RequestDownloadAndInstallStorePackageUpdatesAsync](#) method, you can assign a [Progress](#) handler that is called one time for each step in the download (or download and install) process for each package in this request. The handler receives a [StorePackageUpdateStatus](#) object that provides info about the update package that raised the progress notification. The previous example uses the **PackageDownloadProgress** field of the **StorePackageUpdateStatus** object to display the progress of the download and install process.

Be aware that when you call **RequestDownloadAndInstallStorePackageUpdatesAsync** to download and install package updates in a single operation, the **PackageDownloadProgress** field increases from 0.0 to 0.8 during the download process for a package, and then it increases from 0.8 to 1.0 during the install. Therefore, if you map the percentage shown in your custom progress UI directly to the value of the **PackageDownloadProgress** field, your UI will show 80% when the package is finished downloading and the OS displays the installation dialog. If you want your custom progress UI to display 100% when the package is downloaded and ready to be installed, you can modify your code to assign 100% to your progress UI when the **PackageDownloadProgress** field reaches 0.8.

Download and install package updates silently

Starting in Windows 10, version 1803, you can use the [TrySilentDownloadStorePackageUpdatesAsync](#) and [TrySilentDownloadAndInstallStorePackageUpdatesAsync](#) methods to download and install package updates silently, without displaying a notification UI to the user. This operation will succeed only if the user has enabled the **Update apps automatically** setting in the Store and the user is not on a metered network. Before calling these methods, you can first check the [CanSilentlyDownloadStorePackageUpdates](#) property to determine whether these conditions are currently met.

This code example demonstrates how to use the [GetAppAndOptionalStorePackageUpdatesAsync](#) method to discover all available package updates and then call the [TrySilentDownloadStorePackageUpdatesAsync](#) and [TrySilentDownloadAndInstallStorePackageUpdatesAsync](#) methods to download and install the updates silently.

This code example assumes:

- The code file has a **using** statement for the **Windows.Services.Store** and **System.Threading.Tasks** namespaces.
- The app is a single-user app that runs only in the context of the user that launched the app. For a [multi-user app](#), use the [GetForUser](#) method to get a **StoreContext** object instead of the [GetDefault](#) method.

ⓘ Note

The **IsNowAGoodTimeToRestartApp**, **RetryDownloadAndInstallLater**, and **RetryInstallLater** methods called by the code in this example are placeholder methods that are intended to be implemented as needed according to your own app's design.

C#

```
private StoreContext context = null;

public async Task DownloadAndInstallAllUpdatesInBackgroundAsync()
{
    if (context == null)
    {
        context = StoreContext.GetDefault();
    }

    // Get the updates that are available.
    IReadOnlyList<StorePackageUpdate> storePackageUpdates =
        await context.GetAppAndOptionalStorePackageUpdatesAsync();
```

```

if (storePackageUpdates.Count > 0)
{

    if (!context.CanSilentlyDownloadStorePackageUpdates)
    {
        return;
    }

    // Start the silent downloads and wait for the downloads to
complete.
    StorePackageUpdateResult downloadResult =
        await
context.TrySilentDownloadStorePackageUpdatesAsync(storePackageUpdates);

    switch (downloadResult.OverallState)
    {
        case StorePackageUpdateState.Completed:
            // The download has completed successfully. At this point,
confirm whether your app
            // can restart now and then install the updates (for
example, you might only install
            // packages silently if your app has been idle for a certain
period of time). The
            // IsNowAGoodTimeToRestartApp method is not implemented in
this example, you should
            // implement it as needed for your own app.
            if (IsNowAGoodTimeToRestartApp())
            {
                await InstallUpdate(storePackageUpdates);
            }
            else
            {
                // Retry/reschedule the installation later. The
RetryInstallLater method is not
                // implemented in this example, you should implement it
as needed for your own app.
                RetryInstallLater();
                return;
            }
            break;

            // If the user cancelled the download or you can't perform the
download for some other
            // reason (for example, Wi-Fi might have been turned off and the
device is now on
            // a metered network) try again later. The
RetryDownloadAndInstallLater method is not
            // implemented in this example, you should implement it as
needed for your own app.
        case StorePackageUpdateState.Canceled:
        case StorePackageUpdateState.ErrorLowBattery:
        case StorePackageUpdateState.ErrorWiFiRecommended:
        case StorePackageUpdateState.ErrorWiFiRequired:
        case StorePackageUpdateState.OtherError:
            RetryDownloadAndInstallLater();
            return;
    }
}

```

```

        default:
            break;
    }
}

private async Task InstallUpdate(IReadOnlyList<StorePackageUpdate>
storePackageUpdates)
{
    // Start the silent installation of the packages. Because the packages
    // have already
    // been downloaded in the previous method, the following line of code
    // just installs
    // the downloaded packages.
    StorePackageUpdateResult downloadResult =
        await
context.TrySilentDownloadAndInstallStorePackageUpdatesAsync(storePackageUpda
tes);

    switch (downloadResult.OverallState)
    {
        // If the user cancelled the installation or you can't perform the
        // installation
        // for some other reason, try again later. The RetryInstallLater
        // method is not
        // implemented in this example, you should implement it as needed
        // for your own app.
        case StorePackageUpdateState.Canceled:
        case StorePackageUpdateState.ErrorLowBattery:
        case StorePackageUpdateState.OtherError:
            RetryInstallLater();
            return;
        default:
            break;
    }
}

```

Mandatory package updates

When you create a package submission in Partner Center for an app that targets Windows 10, version 1607 or later, you can [mark the package as mandatory](#) and the date and time on which it becomes mandatory. When this property is set and your app discovers that the package update is available, your app can determine whether the update package is mandatory and alter its behavior until the update is installed (for example, your app can disable features).

ⓘ Note

The mandatory status of a package update is not enforced by Microsoft, and the OS does not provide a UI to indicate to users that a mandatory app update must be installed. Developers are intended to use the mandatory setting to enforce mandatory app updates in their own code.

To mark a package submission as mandatory:

1. Sign in to [Partner Center](#) and navigate to the overview page for your app.
2. Click the name of the submission that contains the package update you want to make mandatory.
3. Navigate to the **Packages** page for the submission. Near the bottom of this page, select **Make this update mandatory** and then choose the day and time on which the package update becomes mandatory. This option applies to all UWP packages in the submission.

For more information, see [Upload app packages](#).

ⓘ Note

If you create a [package flight](#), you can mark the packages as mandatory using a similar UI on the **Packages** page for the flight. In this case, the mandatory package update applies only to the customers who are part of the flight group.

Code example for mandatory packages

The following code example demonstrates how to determine whether any update packages are mandatory. Typically, you should downgrade your app experience gracefully for the user if a mandatory package update does not successfully download or install.

C#

```
private StoreContext context = null;

// Downloads and installs package updates in separate steps.
public async Task DownloadAndInstallAllUpdatesAsync()
{
    if (context == null)
    {
        context = StoreContext.Default();
    }

    // Get the updates that are available.
    IReadOnlyList<StorePackageUpdate> updates =
```

```

        await context.GetAppAndOptionalStorePackageUpdatesAsync();

    if (updates.Count != 0)
    {
        // Download the packages.
        bool downloaded = await DownloadPackageUpdatesAsync(updates);

        if (downloaded)
        {
            // Install the packages.
            await InstallPackageUpdatesAsync(updates);
        }
    }
}

// Helper method for downloading package updates.
private async Task<bool>
DownloadPackageUpdatesAsync(IEnumerable<StorePackageUpdate> updates)
{
    bool downloadedSuccessfully = false;

    IAsyncOperationWithProgress<StorePackageUpdateResult,
    StorePackageUpdateStatus> downloadOperation =
        this.context.RequestDownloadStorePackageUpdatesAsync(updates);

    // The Progress async method is called one time for each step in the
    // download process for each
    // package in this request.
    downloadOperation.Progress = async (asyncInfo, progress) =>
    {
        await
        this.Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
            () =>
            {
                downloadProgressBar.Value = progress.PackageDownloadProgress;
            });
    };

    StorePackageUpdateResult result = await downloadOperation.AsTask();

    switch (result.OverallState)
    {
        case StorePackageUpdateState.Completed:
            downloadedSuccessfully = true;
            break;
        default:
            // Get the failed updates.
            var failedUpdates = result.StorePackageUpdateStatuses.Where(
                status => status.PackageUpdateState !=
                StorePackageUpdateState.Completed);

            // See if any failed updates were mandatory
            if (updates.Any(u => u.Mandatory && failedUpdates.Any(
                failed => failed.PackageFamilyName ==
                u.Package.Id.FamilyName)))

```



```

        {
            // At least one of the updates is mandatory. Perform
whatever actions you
            // want to take for your app: for example, notify the user
and disable
            // features in your app.
            HandleMandatoryPackageError();
        }
        break;
    }

    return downloadedSuccessfully;
}

// Helper method for installing package updates.
private async Task
InstallPackageUpdatesAsync(IEnumerable<StorePackageUpdate> updates)
{
    IAsyncOperationWithProgress<StorePackageUpdateResult,
StorePackageUpdateStatus> installOperation =

this.context.RequestDownloadAndInstallStorePackageUpdatesAsync(updates);

    // The package updates were already downloaded separately, so this
method skips the download
    // operation and only installs the updates; no download progress
notifications are provided.
    StorePackageUpdateResult result = await installOperation.AsTask();

    switch (result.OverallState)
    {
        case StorePackageUpdateState.Completed:
            break;
        default:
            // Get the failed updates.
            var failedUpdates = result.StorePackageUpdateStatuses.Where(
                status => status.PackageUpdateState !=
StorePackageUpdateState.Completed);

            // See if any failed updates were mandatory
            if (updates.Any(u => u.Mandatory && failedUpdates.Any(failed =>
failed.PackageFamilyName == u.Package.Id.FamilyName)))
            {
                // At least one of the updates is mandatory, so tell the
user.
                HandleMandatoryPackageError();
            }
            break;
        }
    }
}

// Helper method for handling the scenario where a mandatory package update
fails to
// download or install. Add code to this method to perform whatever actions
you want

```

```
// to take, such as notifying the user and disabling features in your app.
private void HandleMandatoryPackageError()
{
}
```

Uninstall optional packages

Starting in Windows 10, version 1803, you can use the [RequestUninstallStorePackageAsync](#) or [RequestUninstallStorePackageByStoreIdAsync](#) methods to uninstall an [optional package](#) (including a DLC package) for the current app. For example, if you have an app with content that is installed via optional packages, you might want to provide a UI that enables users to uninstall the optional packages to free up disk space.

The following code example demonstrates how to call [RequestUninstallStorePackageAsync](#). This example assumes:

- The code file has a **using** statement for the **Windows.Services.Store** and **System.Threading.Tasks** namespaces.
- The app is a single-user app that runs only in the context of the user that launched the app. For a [multi-user app](#), use the [GetForUser](#) method to get a **StoreContext** object instead of the [GetDefault](#) method.

C#

```
public async Task UninstallPackage(Windows.ApplicationModel.Package package)
{
    if (context == null)
    {
        context = StoreContext.GetDefault();
    }

    var storeContext = StoreContext.GetDefault();
    IAsyncOperation<StoreUninstallStorePackageResult> uninstallOperation =
        storeContext.RequestUninstallStorePackageAsync(package);

    // At this point, you can update your app UI to show that the package
    // is installing.

    uninstallOperation.Completed += (asyncInfo, status) =>
    {
        StoreUninstallStorePackageResult result =
            uninstallOperation.GetResults();
        switch (result.Status)
        {
            case StoreUninstallStorePackageStatus.Succeeded:
            {
                // Update your app UI to show the package as
```

```

uninstalled.
                break;
            }
        default:
        {
            // Update your app UI to show that the package uninstall
failed.
                break;
            }
        }
    };
}

```

Get download queue info

Starting in Windows 10, version 1803, you can use the [GetAssociatedStoreQueueItemsAsync](#) and [GetStoreQueueItemsAsync](#) methods to get info about the packages that are in the current download and installation queue from the Store. These methods are useful if your app or game supports large optional packages (including DLCs) that can take hours or days to download and install, and you want to gracefully handle the case where a customer closes your app or game before the download and installation process is complete. When the customer starts your app or game again, your code can use these methods to get info about the state of the packages that are still in the download and installation queue so you can display the status of each package to the customer.

The following code example demonstrates how to call [GetAssociatedStoreQueueItemsAsync](#) to get the list of in-progress package updates for the current app and retrieve status info for each package. This example assumes:

- The code file has a **using** statement for the **Windows.Services.Store** and **System.Threading.Tasks** namespaces.
- The app is a single-user app that runs only in the context of the user that launched the app. For a [multi-user app](#), use the [GetForUser](#) method to get a **StoreContext** object instead of the [GetDefault](#) method.

ⓘ Note

The **MarkUpdateInProgressInUI**, **RemoveItemFromUI**, **MarkInstallCompleteInUI**, **MarkInstallErrorInUI**, and **MarkInstallPausedInUI** methods called by the code in this example are placeholder methods that are intended to be implemented as needed according to your own app's design.

```

private StoreContext context = null;

private async Task GetQueuedInstallItemsAndBuildInitialStoreUI()
{
    if (context == null)
    {
        context = StoreContext.GetDefault();
    }

    // Get the Store packages in the install queue.
    IReadOnlyList<StoreQueueItem> storeUpdateItems = await
context.GetAssociatedStoreQueueItemsAsync();

    foreach (StoreQueueItem storeItem in storeUpdateItems)
    {
        // In this example we only care about package updates.
        if (storeItem.InstallKind != StoreQueueItemKind.Update)
            continue;

        StoreQueueItemStatus currentStatus = storeItem.GetCurrentStatus();
        StoreQueueItemState installState =
currentStatus.PackageInstallState;
        StoreQueueItemExtendedState extendedInstallState =
            currentStatus.PackageInstallExtendedState;

        // Handle the StatusChanged event to display current status to the
customer.
        storeItem.StatusChanged += StoreItem_StatusChanged;

        switch (installState)
        {
            // Download and install are still in progress, so update the
status for this
            // item and provide the extended state info. The following
methods are not
            // implemented in this example; you should implement them as
needed for your
            // app's UI.
            case StoreQueueItemState.Active:
                MarkUpdateInProgressInUI(storeItem, extendedInstallState);
                break;
            case StoreQueueItemState.Canceled:
                RemoveItemFromUI(storeItem);
                break;
            case StoreQueueItemState.Completed:
                MarkInstallCompleteInUI(storeItem);
                break;
            case StoreQueueItemState.Error:
                MarkInstallErrorInUI(storeItem);
                break;
            case StoreQueueItemState.Paused:
                MarkInstallPausedInUI(storeItem, installState,

```

```

extendedInstallState);
                break;
            }
        }
    }

    private void StoreItem_StatusChanged(StoreQueueItem sender, object args)
    {
        StoreQueueItemStatus currentStatus = sender.GetCurrentStatus();
        StoreQueueItemState installState = currentStatus.PackageInstallState;
        StoreQueueItemExtendedState extendedInstallState =
currentStatus.PackageInstallExtendedState;

        switch (installState)
        {
            // Download and install are still in progress, so update the status
            // for this
            // item and provide the extended state info. The following methods
            // are not
            // implemented in this example; you should implement them as needed
            // for your
            // app's UI.
            case StoreQueueItemState.Active:
                MarkUpdateInProgressInUI(sender, extendedInstallState);
                break;
            case StoreQueueItemState.Canceled:
                RemoveItemFromUI(sender);
                break;
            case StoreQueueItemState.Completed:
                MarkInstallCompleteInUI(sender);
                break;
            case StoreQueueItemState.Error:
                MarkInstallErrorInUI(sender);
                break;
            case StoreQueueItemState.Paused:
                MarkInstallPausedInUI(sender, installState,
extendedInstallState);
                break;
        }
    }
}

```

Related topics

- [Optional packages and related set authoring](#)

Feedback

Was this page helpful?

 Yes

 No

Devices, sensors, and power

Article • 05/06/2023

Integrate external devices or sensors into your Windows app to provide enhanced experiences for your users.

This section explains how to support various types of devices and sensors and add related features to your app, including enhanced print experiences, motion and orientation sensors, network connections, and battery status.

Enable device capabilities for a Universal Windows Platform app

Article • 05/06/2023

This topic explains how to declare device capabilities in Microsoft Visual Studio to use cameras, microphones, location sensors, and other devices for a Universal Windows Platform (UWP) app.

Specify the device capabilities your app will use

Windows apps require you to specify in the app package manifest when you use certain types of devices. In Visual Studio, you can declare most capabilities by using [Manifest Designer](#) or you can add them manually as described in [How to specify device capabilities in a package manifest \(manually\)](#). This tutorial assumes you're using Manifest Designer.

ⓘ Note

Some types of devices, such as printers, scanners, and sensors, don't need to be declared in the app package manifest.

- In Visual Studio Solution Explorer, double-click the package manifest file, **Package.appxmanifest**.
- Open the **Capabilities** tab.
- Select the device capabilities that your app uses. If you don't see the capability you're looking for in Manifest Designer, add it manually. For more info, see [How to specify device capabilities in a package manifest](#).

Device Capability	Manifest Designer	Description
AllJoyn	✓	Allows AllJoyn-enabled apps and devices on a network to discover and interact with each other. App apps that access APIs in the Windows.Devices.AllJoyn namespace must use this capability.
Blocked Chat Messages	✓	Allows apps to read SMS and MMS messages that have been blocked by the Spam Filter app.
Chat Message Access	✓	Allows apps to read and delete Text Messages. It also allows apps to store chat messages in the system data store.

Device Capability	Manifest Designer	Description
Code Generation	✓	Allows apps to generate code dynamically.
Enterprise Authentication	✓	This capability is subject to the Microsoft Store policy. It provides the capability to connect to enterprise intranet resources that require domain credentials. This capability is not typically needed for most apps.
Internet (Client)	✓	Provides outbound access to the Internet and networks in public places like airports and coffee shops. For example, Intranet networks where the user has designated the network as public. Most apps that require Internet access should use the capability.
Internet (Client & Server)	✓	Provides inbound and outbound access to the Internet and the networks in public places like airports and coffee shops. This capability is a superset of Internet (Client) . Internet (Client) doesn't need to be enabled if this capability is also enabled. Inbound access to critical ports is always blocked.
Location	✓	Provides access to the current location. This is obtained from dedicated hardware like a GPS sensor in the PC, or derived from available network information.
Microphone	✓	Provides access to the microphone's audio feed. This allows the app to record from connected microphones.
Music Library	✓	Provides the capability to add, change, or delete files in the Music Library for the local PC and HomeGroup PCs.
Objects 3D	✓	Provides programmatic access to the user's 3D Objects , allowing the app to enumerate and access all files in the library without user interaction. This capability is typically used in 3D apps and games that need to access the entire 3D Objects library.
Phone Call	✓	Allows apps to access all of the phone lines on the device and perform the following functions: place a call on the phone and show the system dialer without prompting the user; access line-related metadata; access line-related triggers. Allows the user-selected spam filter app to set and check the block list and call origin information.
Pictures Library	✓	Provides the capability to add, change, or delete files in the Pictures Library for the local PC and HomeGroup PCs.
Point of Service	✓	Provides access to Point of Service peripherals. This capability is required to access APIs in the Windows.Devices.PointOfService namespace.

Device Capability	Manifest Designer	Description
Private Networks (Client & Server)	✓	Provides inbound and outbound access to Intranet networks that have an authenticated domain controller, or that the user has designated as either home or work networks. Inbound access to critical ports is always blocked.
Proximity	✓	Provides the capability to connect to devices in close proximity to the PC via near-field communication (NFC). Near-field proximity may be used to send files or communicate with an app on the nearby device.
Removable Storage	✓	Provides the capability to add, change, or delete files on removable storage devices. The app can only access the file types on removable storage that are defined in the manifest using the File Type Associations declaration. The app can't access removable storage on HomeGroup PCs.
Shared User Certificates	✓	This capability is subject to the Microsoft Store policy. It provides the capability to access software and hardware certificates, such as smart card certificates, for validating a user's identity. When related APIs are invoked at runtime, the user must take action (insert card, select certificate, etc.). This capability is not necessary if your app includes a private certificate via a Certificates declaration.
User Account Information	✓	Gives apps the ability to access the user's name and picture. This capability is required to access some APIs in the Windows.System.UserProfile namespace.
Videos Library	✓	Provides the capability to add, change, or delete files in the Videos Library for the local PC and HomeGroup PCs.
VOIP Calling	✓	Allows apps to access the VOIP calling APIs in the Windows.ApplicationModel.Calls namespace.
Webcam	✓	Provides access to the built-in camera or attached webcam's video feed. This allows the app to capture snapshots and movies.
USB		Provides access to custom USB devices. This capability requires child elements. This feature is not supported on Windows Phone.
Human Interface Device (HID)		Provides access to Human Interface Devices (HID). This capability requires child elements. For more info, see How to specify device capabilities for HID .
Bluetooth GATT		Provides access to Bluetooth LE devices through a collection of primary services, included services, characteristics, and descriptors. This capability requires child elements. For more info, see How to specify device capabilities for Bluetooth .

Device Capability	Manifest Designer	Description
Bluetooth RFCOMM		Provides access to APIs that support the Basic Rate/Extended Data Rate (BR/EDR) transport and also lets your UWP app access a device that implements Serial Port Profile (SPP). This capability requires child elements. For more info, see How to specify device capabilities for Bluetooth .

Use the Windows Runtime API for communicating with your device

The following table connects some of the capabilities to Windows Runtime APIs.

Device Capability	API
AllJoyn	Windows.Devices.AllJoyn
Blocked Chat Messages	Windows.ApplicationModel.CommunicationBlocking
Location	See Maps and location overview for more information.
Phone Call	Windows.ApplicationModel.Calls
User Account Information	Windows.System.UserProfile
VOIP Calling	Windows.ApplicationModel.Calls
USB	Windows.Devices.Usb
HID	Windows.Devices.HumanInterfaceDevice
Bluetooth GATT	Windows.Devices.Bluetooth.GenericAttributeProfile
Bluetooth RFCOMM	Windows.Devices.Bluetooth.Rfcomm
Point of Service	Windows.Devices.PointOfService

Enable user mode access to GPIO, I2C, and SPI

Article • 11/15/2023

In Windows 10 and later, APIs are provided with direct access from user mode to general-purpose input/output (GPIO), Inter-Integrated Circuit (I2C), Serial Peripheral Interface (SPI), and universal asynchronous receiver-transmitter (UART). Development boards such as Raspberry Pi 2 expose a subset of these connections, which enable you to extend a base compute module with custom circuitry to address a particular application. These low level buses are usually shared with other critical onboard functions, with only a subset of GPIO pins and buses exposed on headers. To preserve system stability, it is necessary to specify which pins and buses are safe for modification by user mode applications.

This document describes how to specify this configuration in Advanced Configuration and Power Interface (ACPI), and provides tools to validate that the configuration was specified correctly.

Important

The audience for this document is Unified Extensible Firmware Interface (UEFI) and ACPI developers. Some familiarity with ACPI, ACPI Source Language (ASL) authoring, and SpbCx/GpioClx is assumed.

User mode access to low level buses on Windows is plumbed through the existing `GpioClx` and `SpbCx` frameworks. A new driver called *RhProxy*, available on Windows IoT Core and Windows Enterprise, exposes `GpioClx` and `SpbCx` resources to user mode. To enable the APIs, a device node for *rhproxy* must be declared in your ACPI tables with each of the GPIO and SPB resources that should be exposed to user mode. This document walks through authoring and verifying the ASL.

ASL by example

Let's walk through the *rhproxy* device node declaration on Raspberry Pi 2. First, create the ACPI device declaration in the `\_SB` scope.

C++

```
Device(RHPX)
{
    Name(_HID, "MSFT8000")
    Name(_CID, "MSFT8000")
    Name(_UID, 1)
}
```

- _HID – Hardware Id. Set this to a vendor-specific hardware ID.
- _CID – Compatible Id. Must be "MSFT8000".
- _UID – Unique Id. Set to 1.

Next we declare each of the GPIO and SPB resources that should be exposed to user mode. The order in which resources are declared is important because resource indexes are used to associate properties with resources. If there are multiple I2C or SPI busses exposed, the first declared bus is considered the 'default' bus for that type, and will be the instance returned by the `GetDefaultAsync()` methods of

[Windows.Devices.I2c.I2cController](#) and [Windows.Devices.Spi.SpiController](#).

SPI

Raspberry Pi has two exposed SPI buses. SPI0 has two hardware chip select lines and SPI1 has one hardware chip select line. One `SPISerialBus()` resource declaration is required for each chip select line for each bus. The following two `SPISerialBus` resource declarations are for the two chip select lines on SPI0. The `DeviceSelection` field contains a unique value which the driver interprets as a hardware chip select line identifier. The exact value that you put in the `DeviceSelection` field depends on how your driver interprets this field of the ACPI connection descriptor.

ⓘ Note

This article contains references to the term slave—a term that Microsoft doesn't condone, and has stopped using in new products and documentation. When the term is removed from the software, we'll remove it from this article.

C++

```
// Index 0
SPISerialBus(           // SCKL - GPIO 11 - Pin 23
                      // MOSI - GPIO 10 - Pin 19
                      // MISO - GPIO 9  - Pin 21
                      // CE0  - GPIO 8  - Pin 24
    0,                  // Device selection (CE0)
    PolarityLow,        // Device selection polarity
```

```

    FourWireMode,          // wiremode
    0,                     // databit len: placeholder
    ControllerInitiated,   // slave mode
    0,                     // connection speed: placeholder
    ClockPolarityLow,      // clock polarity: placeholder
    ClockPhaseFirst,       // clock phase: placeholder
    "\\_SB.SPI0",          // ResourceSource: SPI bus controller name
    0,                     // ResourceSourceIndex
                           // Resource usage
    )                       // Vendor Data

// Index 1
SPISerialBus(             // SCKL - GPIO 11 - Pin 23
                           // MOSI - GPIO 10 - Pin 19
                           // MISO - GPIO 9 - Pin 21
                           // CE1 - GPIO 7 - Pin 26
    1,                     // Device selection (CE1)
    PolarityLow,           // Device selection polarity
    FourWireMode,          // wiremode
    0,                     // databit len: placeholder
    ControllerInitiated,   // slave mode
    0,                     // connection speed: placeholder
    ClockPolarityLow,      // clock polarity: placeholder
    ClockPhaseFirst,       // clock phase: placeholder
    "\\_SB.SPI0",          // ResourceSource: SPI bus controller name
    0,                     // ResourceSourceIndex
                           // Resource usage
    )                       // Vendor Data

```

How does software know that these two resources should be associated with the same bus? The mapping between bus friendly name and resource index is specified in the DSD:

C++

```
Package(2) { "bus-SPI-SPI0", Package() { 0, 1 }},
```

This creates a bus named "SPI0" with two chip select lines – resource indexes 0 and 1. Several more properties are required to declare the capabilities of the SPI bus.

C++

```
Package(2) { "SPI0-MinClockInHz", 7629 },
Package(2) { "SPI0-MaxClockInHz", 125000000 },
```

The **MinClockInHz** and **MaxClockInHz** properties specify the minimum and maximum clock speeds that are supported by the controller. The API will prevent users from

specifying values outside this range. The clock speed is passed to your SPB driver in the `_SPE` field of the connection descriptor (ACPI section 6.4.3.8.2.2).

C++

```
Package(2) { "SPI0-SupportedDataBitLengths", Package() { 8 }},
```

The **SupportedDataBitLengths** property lists the data bit lengths supported by the controller. Multiple values can be specified in a comma-separated list. The API will prevent users from specifying values outside this list. The data bit length is passed to your SPB driver in the `_LEN` field of the connection descriptor (ACPI section 6.4.3.8.2.2).

You can think of these resource declarations as “templates.” Some of the fields are fixed at system boot while others are specified dynamically at runtime. The following fields of the `SPISerialBus` descriptor are fixed:

- DeviceSelection
- DeviceSelectionPolarity
- WireMode
- SlaveMode
- ResourceSource

The following fields are placeholders for values specified by the user at runtime:

- DataBitLength
- ConnectionSpeed
- ClockPolarity
- ClockPhase

Since SPI1 contains only a single chip select line, a single `SPISerialBus()` resource is declared:

C++

```
// Index 2
SPISerialBus(           // SCKL - GPIO 21 - Pin 40
                      // MOSI - GPIO 20 - Pin 38
                      // MISO - GPIO 19 - Pin 35
                      // CE1 - GPIO 17 - Pin 11
    1,                  // Device selection (CE1)
    PolarityLow,        // Device selection polarity
    FourWireMode,      // wiremode
    0,                  // databit len: placeholder
    ControllerInitiated, // slave mode
    0,                  // connection speed: placeholder
    ClockPolarityLow,   // clock polarity: placeholder
```

```

ClockPhaseFirst,      // clock phase: placeholder
"\_SB.SPI1",          // ResourceSource: SPI bus controller name
0,                    // ResourceSourceIndex
                      // Resource usage
)                      // Vendor Data

```

The accompanying friendly name declaration – which is required – is specified in the DSD and refers to the index of this resource declaration.

C++

```
Package(2) { "bus-SPI-SPI1", Package() { 2 }},
```

This creates a bus named "SPI1" and associates it with resource index 2.

SPI Driver Requirements

- Must use `SpbCx` or be SpbCx-compatible
- Must have passed the [MITT SPI Tests](#)
- Must support 4Mhz clock speed
- Must support 8-bit data length
- Must support all SPI Modes: 0, 1, 2, 3

I2C

Next, we declare the I2C resources. Raspberry Pi exposes a single I2C bus on pins 3 and 5.

C++

```

// Index 3
I2CSerialBus(          // Pin 3 (GPIO2, SDA1), 5 (GPIO3, SCL1)
    0xFFFF,           // SlaveAddress: placeholder
    ,                  // SlaveMode: default to ControllerInitiated
    0,                 // ConnectionSpeed: placeholder
    ,                  // Addressing Mode: placeholder
    "\_SB.I2C1",       // ResourceSource: I2C bus controller name
    ,
    ,
)                       // VendorData

```

The accompanying friendly name declaration – which is required – is specified in the DSD:


```
C++
```

```
Package(2) { "bus-I2C-I2C1", Package() { 3 }},
```

This declares an I2C bus with friendly name "I2C1" that refers to resource index 3, which is the index of the I2CSerialBus() resource that we declared above.

The following fields of the I2CSerialBus() descriptor are fixed:

- SlaveMode
- ResourceSource

The following fields are placeholders for values specified by the user at runtime.

- SlaveAddress
- ConnectionSpeed
- AddressingMode

I2C Driver Requirements

- Must use SpbCx or be SpbCx-compatible
- Must have passed the [MITT I2C Tests](#)
- Must support 7-bit addressing
- Must support 100kHz clock speed
- Must support 400kHz clock speed

GPIO

Next, we declare all the GPIO pins that are exposed to user mode. We offer the following guidance in deciding which pins to expose:

- Declare all pins on exposed headers.
- Declare pins that are connected to useful onboard functions like buttons and LEDs.
- Do not declare pins that are reserved for system functions or are not connected to anything.

The following block of ASL declares two pins – GPIO4 and GPIO5. The other pins are not shown here for brevity. Appendix C contains a sample powershell script which can be used to generate the GPIO resources.

```
C++
```

```
// Index 4 - GPIO 4  
GpioIO(Shared, PullUp, , , , "\\_SB.GPIO", , , , ) { 4 }
```

```
GpioInt(Edge, ActiveBoth, Shared, PullUp, 0, "\\_SB.GPI0",) { 4 }

// Index 6 - GPIO 5
GpioIO(Shared, PullUp, , , , "\\_SB.GPI0", , , , ) { 5 }
GpioInt(Edge, ActiveBoth, Shared, PullUp, 0, "\\_SB.GPI0",) { 5 }
```

The following requirements must be observed when declaring GPIO pins:

- Only memory mapped GPIO controllers are supported. GPIO controllers interfaced over I2C/SPI are not supported. The controller driver is a memory mapped controller if it sets the [MemoryMappedController](#) flag in the [CLIENT_CONTROLLER_BASIC_INFORMATION](#) structure in response to the [CLIENT_QueryControllerBasicInformation](#) callback.
- Each pin requires both a GpioIO and a GpioInt resource. The GpioInt resource must immediately follow the GpioIO resource and must refer to the same pin number.
- GPIO resources must be ordered by increasing pin number.
- Each GpioIO and GpioInt resource must contain exactly one pin number in the pin list.
- The ShareType field of both descriptors must be Shared
- The EdgeLevel field of the GpioInt descriptor must be Edge
- The ActiveLevel field of the GpioInt descriptor must be ActiveBoth
- The PinConfig field
 - Must be the same in both the GpioIO and GpioInt descriptors
 - Must be one of PullUp, PullDown, or PullNone. It cannot be PullDefault.
 - The pull configuration must match the power-on state of the pin. Putting the pin in the specified pull mode from power-on state must not change the state of the pin. For example, if the datasheet specifies that the pin comes up with a pull up, specify PinConfig as PullUp.

Firmware, UEFI, and driver initialization code should not change the state of a pin from its power-on state during boot. Only the user knows what's attached to a pin and therefore which state transitions are safe. The power-on state of each pin must be documented so that users can design hardware that correctly interfaces with a pin. A pin must not change state unexpectedly during boot.

Supported Drive Modes

If your GPIO controller supports built-in pull up and pull down resistors in addition to high impedance input and CMOS output, you must specify this with the optional SupportedDriveModes property.

```
Package (2) { "GPIO-SupportedDriveModes", 0xf },
```

The SupportedDriveModes property indicates which drive modes are supported by the GPIO controller. In the example above, all of the following drive modes are supported. The property is a bitmask of the following values:

Flag Value	Drive Mode	Description
0x1	InputHighImpedance	The pin supports high impedance input, which corresponds to the "PullNone" value in ACPI.
0x2	InputPullUp	The pin supports a built-in pull-up resistor, which corresponds to the "PullUp" value in ACPI.
0x4	InputPullDown	The pin supports a built-in pull-down resistor, which corresponds to the "PullDown" value in ACPI.
0x8	OutputCmos	The pin supports generating both strong highs and strong lows (as opposed to open drain).

InputHighImpedance and OutputCmos are supported by almost all GPIO controllers. If the SupportedDriveModes property is not specified, this is the default.

If a GPIO signal goes through a level shifter before reaching an exposed header, declare the drive modes supported by the SOC, even if the drive mode would not be observable on the external header. For example, if a pin goes through a bidirectional level shifter that makes a pin appear as open drain with resistive pull up, you will never observe a high impedance state on the exposed header even if the pin is configured as a high impedance input. You should still declare that the pin supports high impedance input.

Pin Numbering

Windows supports two pin numbering schemes:

- Sequential Pin Numbering – Users see numbers like 0, 1, 2... up to the number of exposed pins. 0 is the first Gpiolo resource declared in ASL, 1 is the second Gpiolo resource declared in ASL, and so on.
- Native Pin Numbering – Users see the pin numbers specified in Gpiolo descriptors, for example, 4, 5, 12, 13, ...

```
C++
```

```
Package (2) { "GPIO-UseDescriptorPinNumbers", 1 },
```

The **UseDescriptorPinNumbers** property tells Windows to use native pin numbering instead of sequential pin numbering. If the **UseDescriptorPinNumbers** property is not specified or its value is zero, Windows will default to Sequential pin numbering.

If native pin numbering is used, you must also specify the **PinCount** property.

C++

```
Package (2) { "GPIO-PinCount", 54 },
```

The **PinCount** property should match the value returned through the **TotalPins** property in the [CLIENT_QueryControllerBasicInformation](#) callback of the `GpioClx` driver.

Choose the numbering scheme that is most compatible with existing published documentation for your board. For example, Raspberry Pi uses native pin numbering because many existing pinout diagrams use the BCM2835 pin numbers.

MinnowBoardMax uses sequential pin numbering because there are few existing pinout diagrams, and sequential pin numbering simplifies the developer experience because only 10 pins are exposed out of more than 200 pins. The decision to use sequential or native pin numbering should aim to reduce developer confusion.

GPIO Driver Requirements

- Must use `GpioClx`
- Must be on-SOC memory mapped
- Must use emulated ActiveBoth interrupt handling

UART

If your UART driver uses `SerCx` or `SerCx2`, you can use `rhproxy` to expose the driver to user mode. UART drivers that create a device interface of type `GUID_DEVINTERFACE_COMPORT` do not need to use `rhproxy`. The inbox `Serial.sys` driver is one of these cases.

To expose a `SerCx`-style UART to user mode, declare a `UARTSerialBus` resource as follows.

C++

```
// Index 2
UARTSerialBus(          // Pin 17, 19 of JP1, for SIO_UART2
    115200,              // InitialBaudRate: in bits per second
    ,                   // BitsPerByte: default to 8 bits
```

```

, // StopBits: Defaults to one bit
0xfc, // LinesInUse: 8 1-bit flags to declare line
enabled
, // IsBigEndian: default to LittleEndian
, // Parity: Defaults to no parity
, // FlowControl: Defaults to no flow control
32, // ReceiveBufferSize
32, // TransmitBufferSize
"\_SB.URT2", // ResourceSource: UART bus controller name
,
,
,
)

```

Only the ResourceSource field is fixed while all other fields are placeholders for values specified at runtime by the user.

The accompanying friendly name declaration is:

C++

```
Package(2) { "bus-UART-UART2", Package() { 2 }},
```

This assigns the friendly name "UART2" to the controller, which is the identifier users will use to access the bus from user mode.

Runtime Pin Muxing

Pin muxing is the ability to use the same physical pin for different functions. Several different on-chip peripherals, such as an I2C controller, SPI controller, and GPIO controller, might be routed to the same physical pin on a SOC. The mux block controls which function is active on the pin at any given time. Traditionally, firmware is responsible for establishing function assignments at boot, and this assignment remains static through the boot session. Runtime pin muxing adds the ability to reconfigure pin function assignments at runtime. Enabling users to choose a pin's function at runtime speeds development by enabling users to quickly reconfigure a board's pins, and enables hardware to support a broader range of applications than would a static configuration.

Users consume muxing support for GPIO, I2C, SPI, and UART without writing any additional code. When a user opens a GPIO or bus using [OpenPin\(\)](#) or [FromIdAsync\(\)](#), the underlying physical pins are automatically muxed to the requested function. If the pins are already in use by a different function, the [OpenPin\(\)](#) or [FromIdAsync\(\)](#) call will fail. When the user closes the device by disposing the [GpioPin](#), [I2cDevice](#), [SpiDevice](#), or

[SerialDevice](#) object, the pins are released, allowing them to later be opened for a different function.

Windows contains built-in support for pin muxing in the [GpioClx](#), [SpbCx](#), and [SerCx](#) frameworks. These frameworks work together to automatically switch a pin to the correct function when a GPIO pin or bus is accessed. Access to the pins is arbitrated to prevent conflicts among multiple clients. In addition to this built-in support, the interfaces and protocols for pin muxing are general purpose and can be extended to support additional devices and scenarios.

This document first describes the underlying interfaces and protocols involved in pin muxing, and then describes how to add support for pin muxing to GpioClx, SpbCx, and SerCx controller drivers.

Pin Muxing Architecture

This section describes the underlying interfaces and protocols involved in pin muxing. Knowledge of the underlying protocols is not necessarily needed to support pin muxing with GpioClx/SpbCx/SerCx drivers. For details on how to support pin muxing with GpioClx/SpbCx/SerCx drivers, see [Implementing pin muxing support in GpioClx client drivers](#) and [Consuming muxing support in SpbCx and SerCx controller drivers](#).

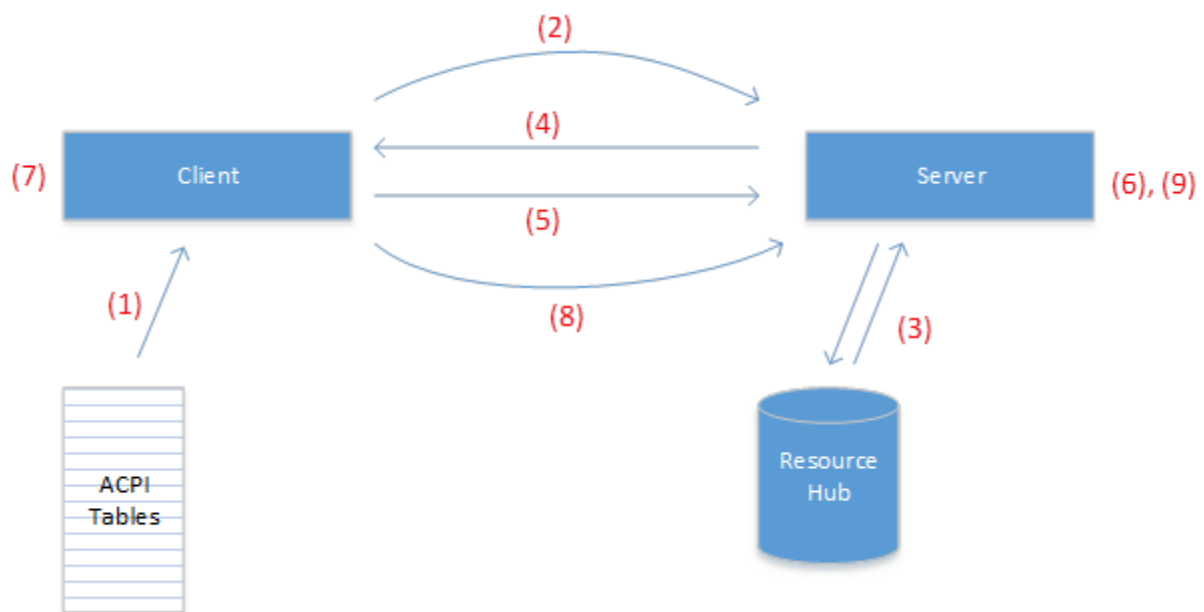
Pin muxing is accomplished by the cooperation of several components.

- Pin muxing servers – these are drivers that control the pin muxing control block. Pin muxing servers receive pin muxing requests from clients via requests to reserve muxing resources (via *IRP_MJ_CREATE*) requests, and requests to switch a pin's function (via **IOCTL_GPIO_COMMIT_FUNCTION_CONFIG_PINS*- requests). The pin muxing server is usually the GPIO driver, since the muxing block is sometimes part of the GPIO block. Even if the muxing block is a separate peripheral, the GPIO driver is a logical place to put muxing functionality.
- Pin muxing clients – these are drivers that consume pin muxing. Pin muxing clients receive pin muxing resources from ACPI firmware. Pin muxing resources are a type of connection resource and are managed by the resource hub. Pin muxing clients reserve pin muxing resources by opening a handle to the resource. To effect a hardware change, clients must commit the configuration by sending an *IOCTL_GPIO_COMMIT_FUNCTION_CONFIG_PINS* request. Clients release pin muxing resources by closing the handle, at which point muxing configuration is reverted to its default state.
- ACPI firmware – specifies muxing configuration with `MsftFunctionConfig()` resources. `MsftFunctionConfig` resources express which pins, in which muxing configuration, are required by a client. `MsftFunctionConfig` resources contain

function number, pull configuration, and list of pin numbers. MsftFunctionConfig resources are supplied to pin muxing clients as hardware resources, which are received by drivers in their PrepareHardware callback similarly to GPIO and SPB connection resources. Clients receive a resource hub ID which can be used to open a handle to the resource.

You must pass the `/MsftInternal` command line switch to `asl.exe` to compile ASL files containing `MsftFunctionConfig()` descriptors since these descriptors are currently under review by the ACPI working committee. For example: `asl.exe /MsftInternal dsdt.asl`

The sequence of operations involved in pin muxing is shown below.



1. The client receives MsftFunctionConfig resources from ACPI firmware in its `EvtDevicePrepareHardware()` callback.
2. The client uses the resource hub helper function `RESOURCE_HUB_CREATE_PATH_FROM_ID()` to create a path from the resource ID, then opens a handle to the path (using `ZwCreateFile()`, `IoGetDeviceObjectPointer()`, or `WdflTargetOpen()`).
3. The server extracts the resource hub ID from the file path using resource hub helper functions `RESOURCE_HUB_ID_FROM_FILE_NAME()`, then queries the resource hub to get the resource descriptor.
4. The server performs sharing arbitration for each pin in the descriptor and completes the `IRP_MJ_CREATE` request.
5. The client issues an `IOCTL_GPIO_COMMIT_FUNCTION_CONFIG_PINS` request on the received handle.

6. In response to `IOCTL_GPIO_COMMIT_FUNCTION_CONFIG_PINS`, the server performs the hardware muxing operation by making the specified function active on each pin.
7. The client proceeds with operations that depend on the requested pin muxing configuration.
8. When the client no longer requires the pins to be muxed, it closes the handle.
9. In response to the handle being closed, the server reverts the pins back to their initial state.

Protocol description for pin muxing clients

This section describes how a client consumes pin muxing functionality. This does not apply to `SerCx` and `SpbCx` controller drivers, since the frameworks implement this protocol on behalf of controller drivers.

Parsing resources

A WDF driver receives `MsftFunctionConfig()` resources in its [EvtDevicePrepareHardware\(\)](#) routine. `MsftFunctionConfig` resources can be identified by the following fields:

C++

```
CM_PARTIAL_RESOURCE_DESCRIPTOR::Type = CmResourceTypeConnection
CM_PARTIAL_RESOURCE_DESCRIPTOR::u.Connection.Class =
CM_RESOURCE_CONNECTION_CLASS_FUNCTION_CONFIG
CM_PARTIAL_RESOURCE_DESCRIPTOR::u.Connection.Type =
CM_RESOURCE_CONNECTION_TYPE_FUNCTION_CONFIG
```

An `EvtDevicePrepareHardware()` routine might extract `MsftFunctionConfig` resources as follows:

C++

```
EVT_WDF_DEVICE_PREPARE_HARDWARE evtDevicePrepareHardware;

_Use_decl_annotations_
NTSTATUS
evtDevicePrepareHardware (
    WDFDEVICE WdfDevice,
    WDFCMRESLIST ResourcesTranslated
)
{
    PAGED_CODE();
```



```

    LARGE_INTEGER connectionId;
    ULONG functionConfigCount = 0;

    const ULONG resourceCount =
WdfCmResourceListGetCount(ResourcesTranslated);
    for (ULONG index = 0; index < resourceCount; ++index) {
        const CM_PARTIAL_RESOURCE_DESCRIPTOR* resDescPtr =
            WdfCmResourceListGetDescriptor(ResourcesTranslated, index);

        switch (resDescPtr->Type) {
        case CmResourceTypeConnection:
            switch (resDescPtr->u.Connection.Class) {
            case CM_RESOURCE_CONNECTION_CLASS_FUNCTION_CONFIG:
                switch (resDescPtr->u.Connection.Type) {
                case CM_RESOURCE_CONNECTION_TYPE_FUNCTION_CONFIG:
                    switch (functionConfigCount) {
                    case 0:
                        // save the connection ID
                        connectionId.LowPart = resDescPtr->
>u.Connection.IdLowPart;
                        connectionId.HighPart = resDescPtr->
>u.Connection.IdHighPart;
                        break;
                    } // switch (functionConfigCount)
                    ++functionConfigCount;
                    break; // CM_RESOURCE_CONNECTION_TYPE_FUNCTION_CONFIG

                } // switch (resDescPtr->u.Connection.Type)
                break; // CM_RESOURCE_CONNECTION_CLASS_FUNCTION_CONFIG
            } // switch (resDescPtr->u.Connection.Class)
            break;
        } // switch
    } // for (resource list)

    if (functionConfigCount < 1) {
        return STATUS_INVALID_DEVICE_CONFIGURATION;
    }
    // TODO: save connectionId in the device context for later use

    return STATUS_SUCCESS;
}

```

Reserving and committing resources

When a client wants to mux pins, it reserves and commits the MsftFunctionConfig resource. The following example shows how a client might reserve and commit MsftFunctionConfig resources.

```

_IRQL_requires_max_(PASSIVE_LEVEL)
NTSTATUS AcquireFunctionConfigResource (
    WDFDEVICE WdfDevice,
    LARGE_INTEGER ConnectionId,
    _Out_ WDFIOTARGET* ResourceHandlePtr
)
{
    PAGED_CODE();

    //
    // Form the resource path from the connection ID
    //
    DECLARE_UNICODE_STRING_SIZE(resourcePath, RESOURCE_HUB_PATH_CHARS);
    NTSTATUS status = RESOURCE_HUB_CREATE_PATH_FROM_ID(
        &resourcePath,
        ConnectionId.LowPart,
        ConnectionId.HighPart);
    if (!NT_SUCCESS(status)) {
        return status;
    }

    //
    // Create a WDFIOTARGET
    //
    WDFIOTARGET resourceHandle;
    status = WdfIoTargetCreate(WdfDevice, WDF_NO_ATTRIBUTES,
&resourceHandle);
    if (!NT_SUCCESS(status)) {
        return status;
    }

    //
    // Reserve the resource by opening a WDFIOTARGET to the resource
    //
    WDF_IO_TARGET_OPEN_PARAMS openParams;
    WDF_IO_TARGET_OPEN_PARAMS_INIT_OPEN_BY_NAME(
        &openParams,
        &resourcePath,
        FILE_GENERIC_READ | FILE_GENERIC_WRITE);

    status = WdfIoTargetOpen(resourceHandle, &openParams);
    if (!NT_SUCCESS(status)) {
        return status;
    }
    //
    // Commit the resource
    //
    status = WdfIoTargetSendIoctlSynchronously(
        resourceHandle,
        WDF_NO_HANDLE,          // WdfRequest
        IOCTL_GPIO_COMMIT_FUNCTION_CONFIG_PINS,
        nullptr,                // InputBuffer
        nullptr,                // OutputBuffer
        nullptr,                // RequestOptions
    );
}

```

```

        nullptr);                // BytesReturned

    if (!NT_SUCCESS(status)) {
        WdfIoTargetClose(resourceHandle);
        return status;
    }

    //
    // Pins were successfully muxed, return the handle to the caller
    //
    *ResourceHandlePtr = resourceHandle;
    return STATUS_SUCCESS;
}

```

The driver should store the `WDFIOTARGET` in one of its context areas so that it can be closed later. When the driver is ready to release the muxing configuration, it should close the resource handle by calling [WdfObjectDelete\(\)](#), or [WdfIoTargetClose\(\)](#) if you intend to reuse the `WDFIOTARGET`.

C++

```
WdfObjectDelete(resourceHandle);
```

When the client closes its resource handle, the pins are muxed back to their initial state, and can now be acquired by a different client.

Protocol description for pin muxing servers

This section describes how a pin muxing server exposes its functionality to clients. This does not apply to `GpioClx` miniport drivers, since the framework implements this protocol on behalf of client drivers. For details on how to support pin muxing in `GpioClx` client drivers, see [Implementing muxing support in GpioClx Client Drivers](#).

Handling `IRP_MJ_CREATE` requests

Clients open a handle to a resource when they want to reserve a pin muxing resource. A pin muxing server receives `IRP_MJ_CREATE` requests by way of a reparse operation from the resource hub. The trailing path component of the `IRP_MJ_CREATE` request contains the resource hub ID, which is a 64-bit integer in hexadecimal format. The server should extract the resource hub ID from the filename using `RESOURCE_HUB_ID_FROM_FILE_NAME()` from `reshub.h`, and send `IOCTL_RH_QUERY_CONNECTION_PROPERTIES` to the resource hub to obtain the `MsftFunctionConfig()` descriptor.

The server should validate the descriptor and extract the sharing mode and pin list from the descriptor. It should then perform sharing arbitration for the pins, and if successful, mark the pins as reserved before completing the request.

Sharing arbitration succeeds overall if sharing arbitration succeeds for each pin in the pin list. Each pin should be arbitrated as follows:

- If the pin is not already reserved, sharing arbitration succeeds.
- If the pin is already reserved as exclusive, sharing arbitration fails.
- If the pin is already reserved as shared,
 - and the incoming request is shared, sharing arbitration succeeds.
 - and the incoming request is exclusive, sharing arbitration fails.

If sharing arbitration fails, the request should be completed with *STATUS_GPIO_INCOMPATIBLE_CONNECT_MODE*. If sharing arbitration succeeds, the request should be completed with *STATUS_SUCCESS*.

Note that the sharing mode of the incoming request should be taken from the MsftFunctionConfig descriptor, not [IrpSp->Parameters.Create.ShareAccess](#).

Handling IOCTL_GPIO_COMMIT_FUNCTION_CONFIG_PINS requests

After the client has successfully reserved a MsftFunctionConfig resource by opening a handle, it can send *IOCTL_GPIO_COMMIT_FUNCTION_CONFIG_PINS* to request the server to perform the actual hardware muxing operation. When the server receives *IOCTL_GPIO_COMMIT_FUNCTION_CONFIG_PINS*, for each pin in the pin list it should

- Set the pull mode specified in the PinConfiguration member of the PNP_FUNCTION_CONFIG_DESCRIPTOR structure into hardware.
- Mux the pin to the function specified by the FunctionNumber member of the PNP_FUNCTION_CONFIG_DESCRIPTOR structure.

The server should then complete the request with *STATUS_SUCCESS*.

The meaning of FunctionNumber is defined by the server, and it is understood that the MsftFunctionConfig descriptor was authored with knowledge of how the server interprets this field.

Remember that when the handle is closed, the server will have to revert the pins to the configuration they were in when *IOCTL_GPIO_COMMIT_FUNCTION_CONFIG_PINS* was received, so the server may need to save the pins' state before modifying them.

Handling IRP_MJ_CLOSE requests

When a client no longer requires a muxing resource, it closes its handle. When a server receives a *IRP_MJ_CLOSE* request, it should revert the pins to the state they were in when *IOCTL_GPIO_COMMIT_FUNCTION_CONFIG_PINS* was received. If the client never sent a *IOCTL_GPIO_COMMIT_FUNCTION_CONFIG_PINS*, no action is necessary. The server should then mark the pins as available with respect to sharing arbitration, and complete the request with *STATUS_SUCCESS*. Be sure to properly synchronize *IRP_MJ_CLOSE* handling with *IRP_MJ_CREATE* handling.

Authoring guidelines for ACPI tables

This section describes how to supply muxing resources to client drivers. Note that you will need Microsoft ASL compiler build 14327 or later to compile tables containing `MsftFunctionConfig()` resources. `MsftFunctionConfig()` resources are supplied to pin muxing clients as hardware resources. `MsftFunctionConfig()` resources should be supplied to drivers that require pin muxing changes, which are typically SPB and serial controller drivers, but should not be supplied to SPB and serial peripheral drivers, since the controller driver handles muxing configuration. The `MsftFunctionConfig()` ACPI macro is defined as follows:

C++

```
MsftFunctionConfig(Shared/Exclusive
    PinPullConfig,
    FunctionNumber,
    ResourceSource,
    ResourceSourceIndex,
    ResourceConsumer/ResourceProducer,
    VendorData) { Pin List }
```

- Shared/Exclusive – If exclusive, this pin can be acquired by a single client at a time. If shared, multiple shared clients can acquire the resource. Always set this to exclusive since allowing multiple uncoordinated clients to access a mutable resource can lead to data races and therefore unpredictable results.
- PinPullConfig – one of
 - PullDefault – use the SOC-defined power-on default pull configuration
 - PullUp – enable pull-up resistor
 - PullDown – enable pull-down resistor
 - PullNone – disable all pull resistors
- FunctionNumber – the function number to program into the mux.
- ResourceSource – The ACPI namespace path of the pin muxing server
- ResourceSourceIndex – set this to 0